Tutorial Pengembangan Website Sekolah

Panduan ini menjelaskan cara mengembangkan website sekolah dengan Next.js, Payload CMS, dan PostgreSQL.

1. Persiapan

Pastikan sudah terpasang:

- Node.js sesuai versi pada `package.json`
- npm
- PostgreSQL atau akses database Supabase
- Git
- Editor kode, misalnya VS Code

Masuk ke folder proyek:

```
cd /home/fycode/Documents/KapitaSelekta
```

Install dependency:

```
npm install
```

2. Konfigurasi Environment

Salin contoh environment:

```
cp .env.example .env
```

Isi `.env` minimal seperti berikut:

```
DATABASE_URL=postgresql://USER:PASSWORD@HOST:5432/DATABASE
PAYLOAD_SECRET=ganti-dengan-secret-random
NEXT_PUBLIC_SERVER_URL=http://localhost:3000
CRON_SECRET=ganti-dengan-secret-cron
PREVIEW_SECRET=ganti-dengan-secret-preview
```

Jangan commit `.env`. File ini berisi password database dan secret aplikasi.

Untuk pengembangan lokal, gunakan database lokal. Untuk database Supabase, gunakan connection string pooler dari Supabase dan jangan menaruh PAT Supabase di `.env` aplikasi.

3. Menjalankan Website

Jalankan server development:

```
npm run dev
```

Buka:

- Website publik: http://localhost:3000
- Admin Payload: http://localhost:3000/admin

Saat pertama kali membuka admin, buat akun administrator. Akun ini digunakan untuk mengelola berita, halaman, media, dan pengguna.

4. Struktur Folder Penting

```
src/
├── app/                Halaman Next.js dan API route
├── collections/        Konfigurasi collection Payload
├── components/         Komponen React yang dapat digunakan ulang
├── fields/             Field Payload yang digunakan bersama
├── globals/            Konfigurasi global seperti Header dan Footer
├── hooks/              Hook Payload dan utilitas terkait
├── migrations/         Migrasi database PostgreSQL
├── plugins/            Konfigurasi plugin Payload
├── payload.config.ts   Konfigurasi utama Payload
└── payload-types.ts    Tipe TypeScript hasil generate Payload
```

Aturan sederhana:

- Halaman publik dikerjakan di `src/app`.
- Model dan field CMS dikerjakan di `src/collections` atau `src/globals`.
- Komponen tampilan diletakkan di `src/components`.
- Jangan mengedit `src/payload-types.ts` secara manual karena file ini dibuat otomatis.

5. Menambah atau Mengubah Collection

Collection adalah tabel dan konfigurasi konten di Payload. Contoh collection berita biasanya berisi judul, slug, isi, gambar sampul, kategori, dan status publikasi.

Langkah umum:

1. Buat atau ubah file collection di `src/collections`.
2. Daftarkan collection di `src/payload.config.ts`.

3. Generate tipe TypeScript.
4. Buat migrasi database.
5. Jalankan migrasi.
6. Jalankan build untuk memastikan aplikasi tetap valid.

Perintah generate tipe:

```
npm run generate:types
```

Buat migrasi baru:

```
npm run payload -- migrate:create nama_migrasi
```

Terapkan migrasi:

```
npm run payload -- migrate
```

Jangan mengandalkan perubahan schema otomatis (`push`) pada database produksi. Semua perubahan schema produksi harus melalui migration yang direview dan disimpan di Git.

6. Alur Membuat Fitur Baru

Gunakan alur berikut untuk setiap fitur:

1. Tentukan data yang diperlukan.
2. Buat atau ubah collection Payload.
3. Buat komponen tampilan.
4. Buat halaman atau route di `src/app`.
5. Tambahkan validasi akses jika data tidak boleh publik.
6. Generate tipe dan migration.
7. Jalankan lint, test, dan build.
8. Periksa tampilan di browser.
9. Commit perubahan dengan pesan yang jelas.

Contoh pembagian fitur:

- Berita: `Posts`, `Categories`, halaman daftar berita, dan halaman detail berita.
- Galeri: `Albums`, `Media`, halaman album, dan galeri foto.
- Struktur organisasi: `Staff`, relasi atasan, dan halaman bagan organisasi.

7. Akses Data dari Next.js

Gunakan Payload Local API di server component atau server-side code ketika aplikasi perlu mengambil data dari Payload. Hindari mengambil data sensitif langsung dari client component.

Untuk data publik:

- Batasi field yang dikirim.
- Gunakan filter status `published`.
- Jangan mengirim password, token, atau data internal ke browser.

Untuk data admin:

- Pastikan route memiliki pemeriksaan autentikasi.
- Gunakan access control Payload.
- Jangan menganggap URL tersembunyi sebagai perlindungan.

8. Pengelolaan Media

Upload media melalui collection `Media` di admin Payload. Periksa hal berikut:

- Alt text wajib diisi untuk gambar penting.
- Ukuran file tidak terlalu besar.
- Nama file mudah dikenali.
- Gambar yang sama tidak diunggah berulang kali.

Saat Supabase Storage sudah dikonfigurasi, media akan disimpan di bucket Supabase. Pastikan bucket, access policy, dan adapter S3 sudah diuji sebelum production.

9. Pemeriksaan Sebelum Commit

Jalankan perintah berikut:

```
npm run generate:types
npm run lint
npm run test:int
npm run build
```

Jika test end-to-end diperlukan:

```
npm run test:e2e
```

Build yang berhasil adalah syarat penting sebelum push. Jika build gagal karena tabel database belum

ada, buat dan jalankan migration terlebih dahulu.

10. Git: Commit dan Push

Periksa perubahan:

```
git status
 git diff --stat
```

Stage file yang memang dibuat atau diubah:

```
git add src/ README.md TUTORIAL-PENGEMBANGAN.md package.json package-lock.json
```

Buat commit:

```
git commit -m "feat: tambahkan fitur berita"
```

Push ke branch aktif:

```
git push
```

Gunakan pesan commit yang menjelaskan perubahan, misalnya:

- `feat: tambahkan collection galeri`
- `fix: perbaiki filter berita published`
- `docs: perbarui tutorial pengembangan`
- `chore: perbarui migration database`

11. Alur Kerja Database Supabase

Untuk perubahan database pada project Supabase:

1. Pastikan project dan environment yang digunakan sudah benar.
2. Uji perubahan di database development atau branch terlebih dahulu.
3. Buat migration Payload dari perubahan collection.
4. Review file migration.
5. Jalankan migration pada target database.
6. Verifikasi tabel dan kolom.
7. Jalankan build aplikasi.

MCP Supabase dapat digunakan untuk memeriksa tabel, migration, dan status project. Operasi SQL yang mengubah data harus diperiksa ulang sebelum dijalankan.

12. Menghubungkan MCP Supabase dengan PAT dan Project Ref

Bagian ini digunakan jika coding agent sudah memiliki kemampuan MCP dan konfigurasi MCP dikelola lewat prompt. Contoh di bawah menggunakan Hermes Agent, tetapi alurnya dapat diadaptasi ke agent lain yang mendukung remote MCP HTTP dan custom header.

Data yang diperlukan

Siapkan dua nilai berikut:

- **PAT Supabase** — Personal Access Token dari halaman Access Tokens Supabase.
- **Project Ref** — ID project Supabase, biasanya terlihat di URL dashboard atau pengaturan project.

PAT adalah credential rahasia. Jangan commit ke Git, jangan memasukkannya ke file proyek, dan jangan menuliskannya di prompt biasa yang tersimpan di riwayat percakapan.

Prompt untuk agent

Kirim prompt berikut kepada coding agent yang Anda gunakan:

```
Konfigurasikan MCP Supabase menggunakan PAT dan Project Ref.

Gunakan server remote HTTP:
https://mcp.supabase.com/mcp?project_ref=PROJECT_REF_SAYA

Atur header:
Authorization: Bearer PAT_SUPABASE_SAYA

Jangan tampilkan PAT di output, log, diff, atau commit.
Simpan credential menggunakan mekanisme secret atau environment variable milik agent.
Setelah konfigurasi, uji koneksi MCP dan laporkan hanya status koneksi,
jumlah tools yang ditemukan, serta nama tools tanpa membocorkan credential.
```

Agent seharusnya meminta PAT melalui input rahasia atau mengambilnya dari environment secret. Jika agent meminta menulis PAT ke file `.env`, pastikan file tersebut hanya berada di luar repository atau sudah masuk `.gitignore`.

Konfigurasi Hermes Agent melalui prompt

Untuk Hermes, prompt yang lebih spesifik:

```
Tambahkan server MCP bernama supabase menggunakan endpoint:
https://mcp.supabase.com/mcp?project_ref=PROJECT_REF_SAYA

Gunakan autentikasi header dengan PAT Supabase yang saya berikan melalui
input rahasia. Jangan mencetak atau menyimpan PAT di repository.
Gunakan full access karena PAT ini memang memiliki izin penuh.
```

```
Setelah itu jalankan verifikasi koneksi MCP dan tampilkan jumlah tools.
```

Jika agent meminta persetujuan konfigurasi, jawab bahwa server perlu diaktifkan dan semua tools yang memang diperlukan boleh didaftarkan. Untuk pekerjaan production, lebih aman memakai `read_only=true` dan membatasi feature group terlebih dahulu.

Cara memverifikasi

Minta agent menjalankan verifikasi berikut:

```
Verifikasi koneksi MCP Supabase sekarang. Tampilkan:
1. nama server;
2. endpoint tanpa PAT;
3. status koneksi;
4. jumlah tools yang ditemukan;
5. daftar nama tools.
Jangan tampilkan token atau header Authorization.
```

Koneksi dianggap berhasil jika agent melaporkan status connected dan tools Supabase berhasil ditemukan. Pada Hermes, konfigurasi dapat diperiksa dengan:

```
hermes mcp list
hermes mcp test supabase
```

Full access dan keamanan

Full access berarti agent dapat melakukan operasi perubahan pada project, bukan hanya membaca data. Dengan PAT full access, agent berpotensi menjalankan SQL, migration, mengubah project, dan melakukan operasi lain sesuai izin PAT.

Gunakan aturan berikut:

- Minta agent menampilkan rencana sebelum menjalankan SQL atau migration.
- Review target project dan query sebelum menyetujui operasi tulis.
- Gunakan project-scoped URL jika hanya satu project yang dibutuhkan.
- Gunakan mode read-only untuk eksplorasi awal atau pekerjaan otomatis.
- Cabut dan buat ulang PAT jika token pernah tercetak di log, chat, screenshot, atau commit.
- Jangan memasukkan PAT ke `README`, source code, `.env.example`, issue, atau pull request.

13. Checklist Fitur Selesai

Sebuah fitur dianggap selesai jika:

- ☐ Collection atau schema sudah dibuat.

- ☐ Access control sudah diperiksa.
- ☐ Halaman publik atau admin sudah dibuat.
- ☐ Tampilan desktop dan mobile sudah diperiksa.
- ☐ Tipe TypeScript sudah di-generate.
- ☐ Migration sudah dibuat dan berhasil dijalankan.
- ☐ Lint dan test terkait berhasil.
- ☐ `npm run build` berhasil.
- ☐ Tidak ada secret yang masuk ke Git.
- ☐ Commit dan push berhasil.